



Docly - Markdown to PDF Converter & PDF Merger

Docly is a command-line tool that converts Markdown files into PDFs using **Puppeteer** for rendering and **Marked.js** for Markdown parsing. It also supports merging multiple PDF files into one and extracting specific pages from PDFs. Syntax highlighting for code blocks is provided by **Highlight.js**, and PDF manipulation is handled by **pdf-lib**.



Features

-  Convert Markdown to PDF effortlessly.
 -  Merge multiple PDFs from a directory into one.
 -  Extract specific pages from existing PDFs.
 -  Supports custom sorting of PDFs during merge (default: natural numeric order).
 -  Clean CLI interface with robust error handling.
 -  **NEW:** Full support for images with relative paths in Markdown files.
 -  **NEW:** Cover page support - render images as full-page front or back covers.
-



Installation

Ensure you have **Node.js 20.8.0 or higher** installed.

1 Install Docly globally:

```
npm install -g docly
```

2 Or, use it directly with `npx` :

```
npx docly --help
```

✂ Usage

Convert a Markdown file to PDF:

```
docly -m input.md -p output.pdf
```

or

```
docly --markdown input.md --pdf output.pdf
```

Image Support

Docly now fully supports images in your Markdown files:

- **Relative paths:** Images referenced with relative paths (e.g., `./images/diagram.png`) are automatically resolved relative to the Markdown file's location
- **Absolute paths:** Full system paths are supported
- **Web URLs:** External images from the web (http/https) work as expected
- **Automatic sizing:** Images are automatically sized to fit within the PDF page width while maintaining aspect ratio
- **Captions:** Regular images display their alt text as captions below the image in a smaller, italic font

Example Markdown with images:

```
# My Document

![Figure 1: Architecture Diagram](./assets/architecture.png)
![User Interface Screenshot](./images/ui-screenshot.png)
 <!-- No caption if alt text is empty -->
![Web image](https://example.com/image.jpg)
```

Note: For best results, ensure your image files exist at the specified paths relative to your Markdown file.

Cover Pages

Docly supports rendering images as full-page covers:

- **Front Cover:** Use `![cover:front](./cover-image.jpg)` to add a front cover
- **Back Cover:** Use `![cover:back](./back-cover.jpg)` to add a back cover
- Cover images are rendered on their own page with no margins
- Images are cropped to fill the entire page (both width and height)
- **Recommendation:** For best results, use cover directives in separate markdown files

Example cover page (cover.md):

```
![cover:front](./front-cover.jpg)
```

Then merge with your main document using the PDF merge feature:

```
docly -m cover.md -p cover.pdf
docly -m document.md -p document.pdf
docly -g ./ -p final-book.pdf
```

Merge multiple PDFs into one:

Provide a directory containing PDF files using the `-g` or `--group` flag. Optionally, specify a sort order with `-s` or `--sort` (`asc` or `desc`). By default, files are sorted in natural ascending order.

```
docly -g ./pdfs -p merged.pdf
```

Example with descending sort:

```
docly -g ./pdfs -p merged.pdf -s desc
```

Extract pages from a PDF:

Extract specific pages from an existing PDF using the `-e` or `--extract` flag along with `-r` or `--range` to specify which pages to extract.

```
docly -e input.pdf -r "1-3" -p output.pdf
```

Supported page range formats:

- Single page: `"5"`
- Page range: `"1-3"`
- Multiple pages: `"1,3,5"`
- Combined: `"1-3,5,7-9"`

Examples:

```
# Extract first page only
docly -e document.pdf -r "1" -p first-page.pdf

# Extract pages 1 through 5
docly -e document.pdf -r "1-5" -p pages1-5.pdf

# Extract pages 1, 3, and 5
docly -e document.pdf -r "1,3,5" -p selected-pages.pdf
```

```
# Extract pages 1-3 and 7-10
```

```
docly -e document.pdf -r "1-3,7-10" -p combined-pages.pdf
```

Programmatic Usage

Docly can also be used as a library in your Node.js applications. All CLI features are available programmatically.

Installation for programmatic use:

```
npm install @brngdsn/docly
```

Import functions:

```
import { convertMarkdownToPdf, mergePdfs, extractPdfPages } from '@brngdsn/docly';
```

API Examples:

Convert Markdown to PDF:

```
const result = await convertMarkdownToPdf({
  markdownPath: './document.md',
  pdfPath: './output.pdf'
});
// result: { success: true, pdfPath: './output.pdf' }
```

Merge PDFs:

```
const result = await mergePdfs({
  dirPath: './pdf-files',
  pdfPath: './merged.pdf',
  sort: 'asc' // optional: 'asc' or 'desc'
});
// result: { success: true, pdfPath: './merged.pdf' }
```

Extract PDF pages:

```
const result = await extractPdfPages({
  inputPath: './source.pdf',
  outputPath: './extracted.pdf',
  pages: '1-3,5,7-10'
```

```
});  
// result: { success: true, outputPath: './extracted.pdf', extractedPages: 7 }
```

Complete example - Create a book:

```
import { convertMarkdownToPdf, mergePdfs } from '@brngdsn/docly';  
  
// Convert cover, content, and back cover  
await convertMarkdownToPdf({  
  markdownPath: './cover.md',  
  pdfPath: './temp/1-cover.pdf'  
});  
  
await convertMarkdownToPdf({  
  markdownPath: './content.md',  
  pdfPath: './temp/2-content.pdf'  
});  
  
await convertMarkdownToPdf({  
  markdownPath: './back.md',  
  pdfPath: './temp/3-back.pdf'  
});  
  
// Merge all PDFs  
await mergePdfs({  
  dirPath: './temp',  
  pdfPath: './final-book.pdf'  
});
```

See the `examples/programmatic-usage.js` file for more detailed examples.

Example Output

Markdown to PDF Conversion:

```
docly -m README.md -p output.pdf
```

Output:

```
Converting "README.md" to "output.pdf"...  
Conversion successful.
```

PDF Merge:

```
docly -g ./pdfs -p "Agentic Engineering.pdf"
```

Output:

```
Merging PDFs from "./pdfs" into "Agentic Engineering.pdf"...  
Merge successful.
```

PDF Page Extraction:

```
docly -e "Full Document.pdf" -r "1-3,5" -p "Selected Pages.pdf"
```

Output:

```
Extracting pages "1-3,5" from "Full Document.pdf" to "Selected Pages.pdf"...  
Extracted 4 page(s) from 10 total pages  
Extraction successful.
```



Dependencies

- [Commander.js](#) – CLI argument parsing.
- [Marked.js](#) – Markdown to HTML conversion.
- [Puppeteer](#) – PDF generation.
- [Highlight.js](#) – Syntax highlighting.
- [pdf-lib](#) – PDF manipulation and merging.

🔧 Development Setup

Clone the repo and install dependencies:

```
git clone https://github.com/brngdsn/docly.git  
cd docly  
npm install
```

Run the CLI locally:

```
node bin/docly.js -m test.md -p test.pdf
```

or for merging PDFs:

```
node bin/docly.js -g ./pdfs -p merged.pdf
```

License

This project is licensed under the **MIT License**.

Contributing

Feel free to open an issue or submit a pull request! 